Módulo 1: Diseño de Sistema

El sistema debe gestionar pólizas individuales y colectivas, incluyendo riesgos, renovaciones, notificaciones e integración con un CORE legado mediante una capa media en WebLogic. Además, las operaciones que cambian estados deben mantenerse sincronizadas con el CORE. El sistema debe estar disponible 24/7 y ser tolerante a fallos.

1. Arquitectura propuesta

Usaría una arquitectura dividida por servicios:

- API Gateway: recibe las solicitudes, valida autenticación, controla acceso y dirige cada petición al servicio correspondiente.
- Servicio de Pólizas: crea, consulta, modifica, renueva y cancela pólizas.
- Servicio de Riesgos: agrega y cancela riesgos, validando que solo se agreguen a pólizas colectivas.
- Adaptador del CORE: es el único componente que se comunica con la capa media en WebLogic y con el CORE de seguros.
- Servicio de Notificaciones: envía correos o SMS cuando se crea o renueva una póliza.
- Base de datos: almacena pólizas, riesgos, estados, auditoría y eventos pendientes.

Para las operaciones de escritura, primero se validan las reglas de negocio. Después se consume el servicio agnóstico de edición por medio de WebLogic para actualizar el CORE. Cuando la operación es exitosa, se actualiza el estado local y se genera el evento de notificación.

2. Patrones de arquitectura

API Gateway:
Permite tener un único punto de entrada para los clientes. También facilita manejar seguridad, versionamiento, límites de consumo y direccionamiento hacia los servicios.

Arquitectura hexagonal:
Permite separar la lógica del negocio de la base de datos y de sistemas externos como WebLogic o el CORE. Así, si cambia la forma de conexión al CORE, no es necesario modificar toda la lógica de pólizas.

Arquitectura orientada a eventos:
Las acciones como creación o renovación pueden generar eventos. El servicio de notificaciones los procesa sin bloquear la operación principal. También facilita agregar otros consumidores en el futuro.

3. Modelo de datos principal

Póliza

- id
- tipo: INDIVIDUAL o COLECTIVA
- estado
- fecha de inicio
- fecha de fin
- número de meses de vigencia
- canon mensual
- prima
- tomador
- asegurado
- beneficiario
- porcentaje de IPC aplicado

Riesgo

- id
- id de la póliza
- información del inmueble
- arrendatario
- arrendador
- canon del inmueble
- estado del riesgo

Una póliza colectiva puede tener varios riesgos. Una póliza individual solamente puede tener un riesgo.

También tendría una tabla de eventos pendientes para guardar las operaciones que todavía no se han podido enviar al CORE o los eventos de notificación que deben ser procesados.

4. Manejo de aspectos técnicos

Escalabilidad

- Los servicios serían stateless para poder levantar varias instancias.
- El API Gateway distribuiría las solicitudes.
- Se crearían índices para búsquedas por tipo, estado e id de póliza.
- Las notificaciones se procesarán mediante una cola.
- Si el volumen aumenta, se podrían usar réplicas de lectura en la base de datos.

Logs y observabilidad

Usaría logs estructurados con información como:

- id de la póliza
- id del riesgo
- operación realizada
- resultado
- tiempo de respuesta

- identificador de correlación

También mediría errores, tiempos de respuesta, operaciones pendientes con el CORE, renovaciones realizadas y mensajes de notificación fallidos.

Tolerancia a fallos

- Timeouts para las llamadas al CORE.
- Reintentos controlados con espera progresiva.
- Si el CORE no responde, la operación queda pendiente y se reintenta posteriormente.
- Se mantendría una auditoría de los cambios realizados.

Versionamiento de APIs

propondría utilizar un flujo de ramas basado en Git Flow, con ramas permanentes **`production`** y **`testing`**, ramas temporales **`feature/*, release/*`** y **`hotfix/*.`**

La rama **`production`** representa exactamente el código desplegado en producción y estará protegida contra pushes directos. La rama **`testing`** representa la versión de pruebas que se encuentra en integración y QA.

Las nuevas funcionalidades se desarrollarán desde **`testing`** en ramas **`feature/*`**, por ejemplo **`feature/POL-145-renovar-poliza.`** Una vez finalizadas, se integrarán mediante Pull Request a testing, donde deberán ejecutarse los pipelines de CI, pruebas unitarias, pruebas de integración, análisis estático y validaciones de seguridad.

Cuando una versión esté lista para producción se creará una rama **`release/x.y.z`**, destinada únicamente a correcciones, configuración y preparación final de la versión. Una vez aprobada, se integrará a **`production`**.

Cada versión desplegada en producción se identificará mediante un tag siguiendo Semantic Versioning:

MAJOR.MINOR.PATCH

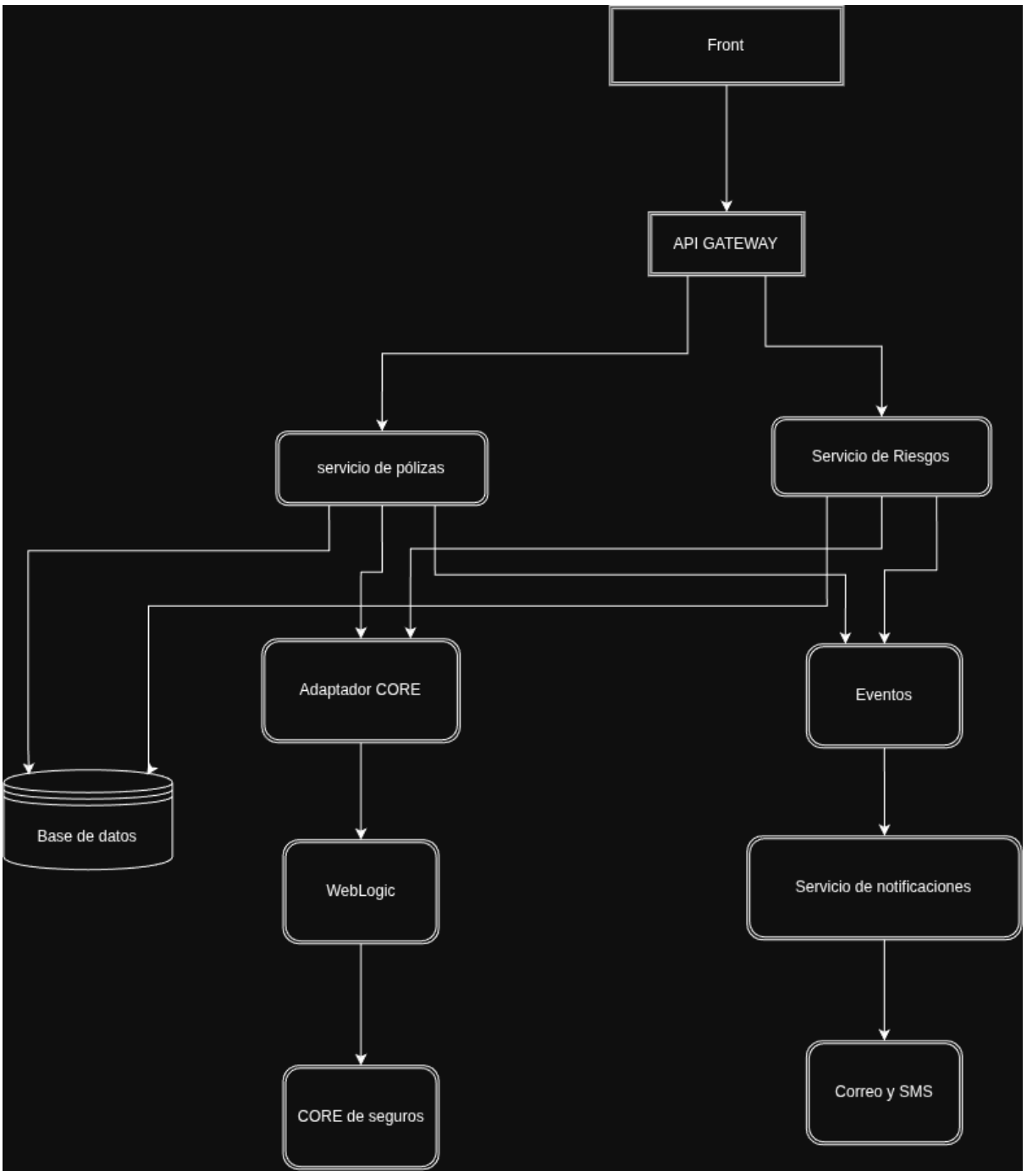
Ejemplos:

v1.0.0 > primera versión estable.

v1.1.0 > nueva funcionalidad compatible.

v1.1.1 > corrección pequeña de código.

5. Diagrama de componentes



módulo 2 Prueba Técnica Práctica

https://github.com/Montyer930/prueba_tecnica

Módulo 3: Base de Datos

1. Crear índices

Crearía un índice para el campo utilizado en el filtro y otro para el campo del `JOIN`:

```sql
CREATE INDEX idx_customers_country_id
ON customers(country, customer_id);

CREATE INDEX idx_orders_customer_id
ON orders(customer_id);
```

El primer índice ayuda a encontrar rápidamente los clientes de México y el segundo permite buscar sus pedidos sin recorrer toda la tabla **`orders`**.

2. Revisar el plan de ejecución

Antes de realizar cambios, utilizaría EXPLAIN por ejemplo

Con esto buscaría principalmente:

- Full Table Scan sobre customers o orders.
- Índices que no estén siendo utilizados.
- Cantidad de registros que se están procesando.
- Tipo de JOIN utilizado.
- Costo estimado de cada operación.

Por ejemplo, si el plan muestra que se están recorriendo los 10 millones de registros de orders, sería una señal clara de que el acceso por customer_id debe optimizarse.

Después de crear los índices volvería a ejecutar el plan para comprobar que realmente fueron utilizados y que el costo de la consulta disminuyó.

3. Usar una vista materializada o tabla de consulta

Si la consulta se ejecuta muchas veces y los datos no cambian constantemente, evaluaría crear una vista materializada con la información ya filtrada o una tabla de apoyo para reportes.

Esto evitaría realizar el `JOIN` completo en cada ejecución. La vista tendría que actualizarse cuando cambien los clientes, los pedidos o el país del cliente.

Además, aclararía que la consulta ya está usando un `INNER JOIN`:

```sql
JOIN customers c
```

Es equivalente a:

```sql
INNER JOIN customers c
```

Por eso cambiar esa parte no tendría un efecto importante en el rendimiento. La mejora principal vendría de los índices, las estadísticas y la forma en que el motor está ejecutando la consulta.

Módulo 4: Git y GitHub

Usaría `git cherry-pick`, porque permite traer un commit específico de otra rama sin incorporar todos los cambios posteriores.

```
ejemplo:

git cherry-pick a45bc12
```

¿Por qué cherry-pick?

Porque **`merge`** traería toda la línea de cambios de **`main`**:

Módulo 5: Liderazgo Técnico

1. Prioridades en las primeras dos semanas

1. Revisar los incidentes recientes y corregir las causas más importantes.
2. Definir con negocio el alcance mínimo de la funcionalidad.
3. Priorizar la deuda técnica que afecta la estabilidad.
4. Establecer pull requests, revisión de código y pruebas obligatorias.
5. Acompañar a los desarrolladores junior con apoyo de los seniors.

2. Organización del equipo

Formaría parejas de trabajo, combinando desarrolladores senior y junior. Dos parejas trabajan en la nueva funcionalidad, una en estabilidad y otra en deuda técnica y automatización. Haría reuniones cortas para revisar avances y bloqueos.

3. Métricas

- Número de incidentes y tiempo de solución.
- Errores encontrados después de una entrega.
- Tiempo de desarrollo y frecuencia de despliegues.
- Tiempo de revisión de código.
- Deuda técnica solucionada.

4. Prácticas obligatorias

- Todo cambio debe pasar por un pull request.
- Revisión de código antes de integrar.
- Pruebas automatizadas para las reglas importantes.
- Validaciones automáticas en el pipeline.
- Logs, documentación y posibilidad de rollback.

5. Presión del negocio

Definiría un MVP con las funciones más importantes y dejaría las mejoras secundarias para otra etapa. Mantendría las revisiones, pruebas y controles de seguridad. También informaría periódicamente los avances, riesgos y posibles cambios de alcance.